new idea

Query Modifiers

LIMIT, OFFSET

teacher

id	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000.00
83821	Brandt	Comp. Sci.	92000.00
12121	Wu	Finance	90000.00
33456	Gold	Physics	87000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00
45565	Katz	Comp. Sci.	75000.00
76766	Crick	Biology	72000.00
10101	Srinivasan	Comp. Sci.	65000.00
58583	Califieri	History	62000.00
32343	El Said	History	60000.00
15151	Mozart	Music	40000.00

`LIMIT` / `OFFSET` are **not** standard SQL!

```
SELECT *
FROM teacher
ORDER BY SALARY DESC
LIMIT 3;
```

id	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000.00
83821	Brandt	Comp. Sci.	92000.00
12121	Wu	Finance	90000.00

```
SELECT *
FROM teacher
ORDER BY SALARY DESC
OFFSET 6;
```

id	*name*	*dept_name*	*salary*
45565	Katz	Comp. Sci.	75000.00
76766	Crick	Biology	72000.00
10101	Srinivasan	Comp. Sci.	65000.00
58583	Califieri	History	62000.00
32343	El Said	History	60000.00
15151	Mozart	Music	40000.00

without ORDER BY, LIMIT returns **arbitrary** rows. So always pair them!

teacher

id	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000.00
83821	Brandt	Comp. Sci.	92000.00
12121	Wu	Finance	90000.00
33456	Gold	Physics	87000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00
45565	Katz	Comp. Sci.	75000.00
76766	Crick	Biology	72000.00
10101	Srinivasan	Comp. Sci.	65000.00
58583	Califieri	History	62000.00
32343	El Said	History	60000.00
15151	Mozart	Music	40000.00

LIMIT x OFFSET y:
give x results, starting after y top results

```
SELECT *
FROM teacher
ORDER BY SALARY DESC
LIMIT 3 OFFSET 5;
```



id	*name*	*dept_name*	*salary*
98345	Kim	Elec. Eng.	80000.00
45565	Katz	Comp. Sci.	75000.00
76766	Crick	Biology	72000.00

Logical Order of Processing

FROM ⟶ WHERE ⟶ GROUP BY ⟶ AGGREGATIONS ⟶ HAVING ⟶ SELECT ⟶ DISTINCT ⟶ ORDER BY ⟶ LIMIT OFFSET
⟶ aggregations available from here onwards

new idea

ARRAYS

In PostgreSQL, an ARRAY is a list (ordered) of elements with **same data type**.

```
SELECT ARRAY['a', 'b', 'c', '3'];
```

↓

array
{a,b,c,3}

`ARRAY['a', 3]` gives an error.

`ARRAY['a', '3']` is correct.

```
ALTER TABLE teacher ADD COLUMN specialties text[];
```

teacher

id	*name*	*dept_name*	*salary*	*specialties*
10101	Srinivasan	Comp. Sci.	65000.00	
12121	Wu	Finance	90000.00	
15151	Mozart	Music	40000.00	
22222	Einstein	Physics	95000.00	
32343	El Said	History	60000.00	
33456	Gold	Physics	87000.00	
45565	Katz	Comp. Sci.	75000.00	
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
76766	Crick	Biology	72000.00	
83821	Brandt	Comp. Sci.	92000.00	
98345	Kim	Elec. Eng.	80000.00	

Or directly create a new table:

```
create table teacher
  (ID   varchar(5),
   name   varchar(20),
   dept_name  varchar(20),
   salary   numeric(8,2),
   specialties  text[],
   primary key (ID),
   foreign key (dept_name) references department
  );
```

setting ARRAY values

```
UPDATE teacher
SET specialties = ARRAY['python', 'sql', 'complexity']
WHERE id = '10101';
```

teacher

id	*name*	*dept_name*	*salary*	*specialties*
12121	Wu	Finance	90000.00	
15151	Mozart	Music	40000.00	
22222	Einstein	Physics	95000.00	
32343	El Said	History	60000.00	
33456	Gold	Physics	87000.00	
45565	Katz	Comp. Sci.	75000.00	
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
76766	Crick	Biology	72000.00	
83821	Brandt	Comp. Sci.	92000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity

```
ALTER TABLE teacher ADD COLUMN specialties text[];

UPDATE teacher SET specialties = '{python,sql,complexity}' WHERE ID = '10101';
UPDATE teacher SET specialties = '{fin1,fin2}'             WHERE ID = '12121';
UPDATE teacher SET specialties = '{mus1,mus2,mus3,mus4}'   WHERE ID = '15151';
UPDATE teacher SET specialties = '{phys1,phys2,phys3}'     WHERE ID = '22222';
UPDATE teacher SET specialties = '{his1,his2,his3}'        WHERE ID = '32343';
UPDATE teacher SET specialties = '{phys2,phys1,phys4}'     WHERE ID = '33456';
UPDATE teacher SET specialties = '{sql,complexity}'        WHERE ID = '45565';
UPDATE teacher SET specialties = '{bio6,bio4}'             WHERE ID = '76766';
UPDATE teacher SET specialties = '{sql,algorithms}'        WHERE ID = '83821';
```

two ways to write an array:
`ARRAY['python', 'sql']` or `{'python','sql'}`

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

Arrays are suited for a **single** attribute with several values.

For **relational** data, better to instead have separate tables:

table as before

```
create table teacher
  (ID   varchar(5),
   name   varchar(20),
   dept_name  varchar(20),
   salary   numeric(8,2),
   primary key (ID),
   foreign key (dept_name) references department
  );
```

new speciality table

```
create table teacher_specialty
  (ID   varchar(5),
   specialty  varchar(20),
   primary key (ID, specialty),
   foreign key (ID) references teacher (ID)
  );
```

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

selecting value in ARRAY

```
SELECT name, specialties[1]
FROM teacher;
```

→

name	*specialties*
Califieri	
Singh	
Kim	
Srinivasan	python
Wu	fin1
Mozart	mus1
Einstein	phys1
El Said	his1
Gold	phys2
Katz	sql
Crick	bio6
Brandt	sql

index on `null` gives `null`

Note: first index is 1, not 0

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

selecting multiple values in ARRAY

```
SELECT name, specialties[1:2]
FROM teacher;
```

name	*specialties*
Califieri	
Singh	
Kim	
Srinivasan	python,sql
Wu	fin1,fin2
Mozart	mus1,mus2
Einstein	phys1,phys2
El Said	his1,his2
Gold	phys2,phys1
Katz	sql,complexity
Crick	bio6,bio4
Brandt	sql,algorithms

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

creating ARRAY directly in SELECT

```
SELECT ARRAY['first', 'second', name]
FROM teacher;
```

array
first,second,Califieri
first,second,Singh
first,second,Kim
first,second,Srinivasan
first,second,Wu
first,second,Mozart
first,second,Einstein
first,second,"El Said"
first,second,Gold
first,second,Katz
first,second,Crick
first,second,Brandt

out-of-index gives `null`

```
SELECT (ARRAY['first', 'second'])[5];
```

array
null

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

search for exact match for any ARRAY value

```
SELECT *
FROM teacher
WHERE 'phys1' = ANY(specialties);
```



id	*name*	*dept_name*	*salary*	*specialties*
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
33456	Gold	Physics	87000.00	phys2,phys1,phys4

unfolding ARRAY into multiple rows

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

```
SELECT name, unnest(specialties)
FROM teacher;
```



name	*unnest*
Srinivasan	python
Srinivasan	sql
Srinivasan	complexity
Wu	fin1
Wu	fin2
Mozart	mus1
Mozart	mus2
Mozart	mus3
Mozart	mus4
Einstein	phys1
Einstein	phys2
Einstein	phys3
El Said	his1
El Said	his2
El Said	his3
Gold	phys2
Gold	phys1
Gold	phys4
Katz	sql
Katz	complexity
Crick	bio6
Crick	bio4
Brandt	sql
Brandt	algorithms

unnest: converts an `array` into a `table`

unnest: converts an `array` into a `table`

```
SELECT *
FROM unnest( array['abc','def'] ) as mytable(attrib1) ;
```

attrib1
abc
def

another way to write it:

```
SELECT unnest( array['abc','def'] ) as attrib1 ;
```

attrib1
abc
def

```
SELECT name, spec
FROM teacher, unnest(teacher.specialties) AS t(spec);
```

FROM: evaluated once **per teacher row**

from each row of teacher, a row for each specialty of that teacher

name	*spec*
Srinivasan	python
Srinivasan	sql
Srinivasan	complexity
Wu	fin1
Wu	fin2
Mozart	mus1
Mozart	mus2
Mozart	mus3
Mozart	mus4
Einstein	phys1
Einstein	phys2
Einstein	phys3
El Said	his1
El Said	his2
El Said	his3
Gold	phys2
Gold	phys1
Gold	phys4
Katz	sql
Katz	complexity
Crick	bio6
Crick	bio4
Brandt	sql
Brandt	algorithms

```
SELECT name, l
FROM teacher, unnest(ARRAY['x', 'y']) AS t(l);
```

name	*l*
Califieri	x
Singh	x
Kim	x
Srinivasan	x
Wu	x
Mozart	x
Einstein	x
El Said	x
Gold	x
Katz	x
Crick	x
Brandt	x
Califieri	y
Singh	y
Kim	y
Srinivasan	y
Wu	y
Mozart	y
Einstein	y
El Said	y
Gold	y
Katz	y
Crick	y
Brandt	y

does **not** reference `teacher` → **cross join**

RULE

a function in `FROM` that reads another `FROM` item's columns is **LATERAL** (per row)

One that doesn't is **cross-joined**.

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

ARRAY_AGG: combining all selected values into a single ARRAY

```
SELECT ARRAY_AGG(name)
FROM teacher;
```

array_agg
Califieri,Singh,Kim,Srinivasan,Wu,Mozart,Einstein,"El Said",Gold,Katz,Crick,Brandt

without `ORDER BY`, the order of `ARRAY_AGG` is **not guaranteed**

```
SELECT ARRAY_AGG(name ORDER BY name DESC)
FROM teacher;
```

array_agg
Wu,Srinivasan,Singh,Mozart,Kim,Katz,Gold,"El Said",Einstein,Crick,Califieri,Brandt

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

ARRAY_AGG: aggregating all selected values into a single ARRAY

```
SELECT ARRAY_AGG(course_id || ': ' || title)
FROM course
WHERE dept_name = 'Comp. Sci.';
```

array_agg
"CS-101: Intro. to Computer Science","CS-190: Game Design","CS-315: Robotics","CS-319: Image Processing","CS-347: Database System Concepts"

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

```
SELECT ARRAY (SELECT name FROM teacher WHERE dept_name = 'Comp. Sci.');
```



array
{Srinivasan,Katz,Brandt}

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

converting all ARRAY elements into a single text using `array_to_string`

```
SELECT array_to_string(specialties, '-')
FROM teacher;
```



array_to_string
python–sql–complexity
fin1–fin2
mus1–mus2–mus3–mus4
phys1–phys2–phys3
his1–his2–his3
phys2–phys1–phys4
sql–complexity
bio6–bio4
sql–algorithms

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

partial search in ARRAY using `array_to_string`

```
SELECT *
FROM teacher
WHERE array_to_string(specialties, ',')  LIKE 'sql%';
```

id	*name*	*dept_name*	*salary*	*specialties*
45565	Katz	Comp. Sci.	75000.00	{sql,complexity}
83821	Brandt	Comp. Sci.	92000.00	{sql,algorithms}

Note: only looks at the **start** of the joined string.

E.g.: `array_to_string(ARRAY['sql','python'], ',') LIKE 'p%'` is **false**!

teacher

id	*name*	*dept_name*	*salary*	*specialties*
58583	Califieri	History	62000.00	
76543	Singh	Finance	80000.00	
98345	Kim	Elec. Eng.	80000.00	
10101	Srinivasan	Comp. Sci.	65000.00	python,sql,complexity
12121	Wu	Finance	90000.00	fin1,fin2
15151	Mozart	Music	40000.00	mus1,mus2,mus3,mus4
22222	Einstein	Physics	95000.00	phys1,phys2,phys3
32343	El Said	History	60000.00	his1,his2,his3
33456	Gold	Physics	87000.00	phys2,phys1,phys4
45565	Katz	Comp. Sci.	75000.00	sql,complexity
76766	Crick	Biology	72000.00	bio6,bio4
83821	Brandt	Comp. Sci.	92000.00	sql,algorithms

partial search in ARRAY using unnest

```
SELECT *
FROM teacher
WHERE EXISTS (SELECT 1
              FROM unnest(specialties) as data
              WHERE data LIKE 'sql%');
```

now a complete search through entire array

id	*name*	*dept_name*	*salary*	*specialties*
10101	Srinivasan	Comp. Sci.	65000.00	{python,sql,complexity}
45565	Katz	Comp. Sci.	75000.00	{sql,complexity}
83821	Brandt	Comp. Sci.	92000.00	{sql,algorithms}

```
array_append(a, x) = a || x,  array_cat(a, b) = a || b

a && b:  true if share at least one element
a @> b:  true if all elements of b are in a
cardinality(a):  number of elements in a
array_position(a, x):  index of first x (or NULL)
```

```
SELECT ARRAY['python','sql'] && ARRAY['sql','ml'],
       ARRAY['python','sql'] @> ARRAY['python'];
```

?column?	*?column?*
t	t

```
SELECT cardinality(ARRAY['a','b','c']),
       array_position(ARRAY['a','b','c'], 'b'),
       array_position(ARRAY['a','b','c'], 'x');
```

cardinality	*array_position*	*array_position*
3	2	


```
array_append(a, x) = a || x,  array_cat(a, b) = a || b

a && b:  true if share at least one element
a @> b:  true if all elements of b are in a
cardinality(a):  number of elements in a
array_position(a, x):  index of first x (or NULL)
```


```
SELECT array_append(ARRAY['a', 'b'], 'c'),
       array_cat(ARRAY['a', 'b'], ARRAY['c', 'd']),
       ARRAY['a', 'b'] || ARRAY['c', 'd'];
```

array_append	*array_cat*	*?column?*
{a,b,c}	{a,b,c,d}	{a,b,c,d}

Many functions and functionalities available for use with ARRAYS

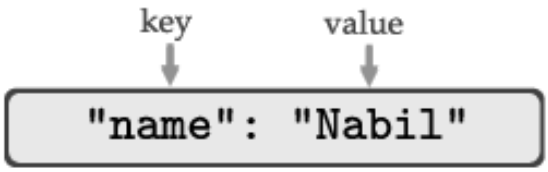
not very convenient—use only for special reasons (tags, etc.) where do not need to unnest often.

Operator	Description	Example	Result
=	equal	ARRAY[1.1,2.1,3.1]::int[] = ARRAY[1,2,3]	t
<>	not equal	ARRAY[1,2,3] <> ARRAY[1,2,4]	t
<	less than	ARRAY[1,2,3] < ARRAY[1,2,4]	t
>	greater than	ARRAY[1,4,3] > ARRAY[1,2,4]	t
<=	less than or equal	ARRAY[1,2,3] <= ARRAY[1,2,3]	t
>=	greater than or equal	ARRAY[1,4,3] >= ARRAY[1,4,3]	t
@>	contains	ARRAY[1,4,3] @> ARRAY[3,1]	t
<@	is contained by	ARRAY[2,7] <@ ARRAY[1,7,4,2,6]	t
&&	overlap (have elements in common)	ARRAY[1,4,3] && ARRAY[2,1]	t
||	array-to-array concatenation	ARRAY[1,2,3] || ARRAY[4,5,6]	{1,2,3,4,5,6}
||	array-to-array concatenation	ARRAY[1,2,3] || ARRAY[[4,5,6],[7,8,9]]	{{1,2,3},{4,5,6},{7,8,9}}
||	element-to-array concatenation	3 || ARRAY[4,5,6]	{3,4,5,6}
||	array-to-element concatenation	ARRAY[4,5,6] || 7	{4,5,6,7}

new idea

JSON format

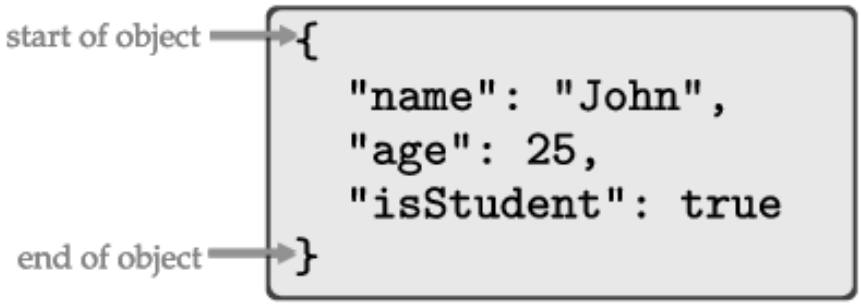
basic building block: a pair



keys: always `strings` in double quotes

values: can be `strings`, `numbers`, `booleans`, `objects`, `arrays`, or `null`

object: a collection of key-value pairs



pairs separated by commas

order of pairs doesn't matter ... it is a *set* (unordered)

as objects can be a **value**, we can have nested structures

```
{
  "user": {
    "firstName": "Maria",
    "lastName": "Garcia",
    "address": {
      "street": "123 Main St",
      "city": "Boston",
      "zipCode": "02101"
    }
  }
}
```

Recursive structure

```
{ ... }
```

The entire file can be one unnamed root object

keys must be unique (*inside the same object*)

```
{
  "name" : "Nabil",
  "name" : "also nabil"
}
```

✗

```
{
  "name" : "Nabil",
  "property": {
      "name" : "also nabil"
   }
}
```

✓

arrays are an **ordered list**

```
{
  "fruits": ["apple", "banana", "orange"],
  "testScores": [95, 87, 92, 78],
  "mixedArray": [1, "hello", true, null]
}
```

Unlike SQL, JSON arrays can mix data types

Arrays can be put in objects

```
{
  "classroom": {
    "name": "Math 101",
    "students": ["Alice", "Bob", "Charlie"],
    "schedule": ["Monday", "Wednesday", "Friday"]
  }
}
```

Objects can be put in arrays

```
{
  "students": [
    {
      "id": 1,
      "name": "Alice",
      "grade": "A"
    },
    {
      "id": 2,
      "name": "Bob",
      "grade": "B"
    }
  ]
}
```

Number of objects: 3 Number of pairs: 7 Number of arrays: 1

```
{
  "user": {
    "profile": {
      "personal": {
        "name": "John Doe",
        "age": 30
      },
      "professional": {
        "title": "Developer",
        "company": "Tech Corp"
      }
    }
  }
}
```

```
{
  "user": { ... }
}
```

main object with one key-value pair.

```
"user": {
  "profile": { ... }
}
```

value of `user` is an object containing one key-value pair

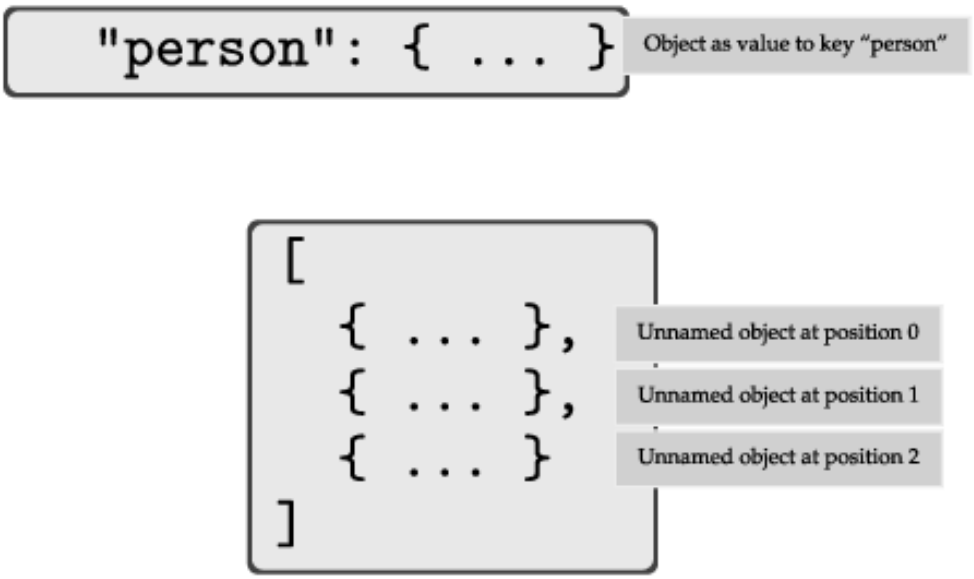
Number of objects: 5 — `Root, User, Profile, Personal, Professional`

Number of pairs: 8

Number of arrays: 0

Nesting Levels 4: Root → User → Profile → Personal/Professional

Objects associated with a key when in key-value pairs
In arrays, associated with an index



A JSON file = exactly one value:
a string, a number, a boolean, `null`, an object `{ }` or an array `[ ]`.

object: contains *pairs only*, each of format `"key": value`

```
{"name": "John", "age": 30}
```

array: any kind and can be recursive

```
["hello", 1, true, {"k": 2}, [3, 4]]
```

invalid:

`{ "hello" }` a bare value in an object ⟶ it needs a key
`"a" "b"` two values ⟶ a file holds exactly one
`"a": 1` a pair without its object ⟶ pairs need `{ }`

Some valid JSON files

```
"John"
```

✓

```
{
  "name": "John",
  "age": 30
}
```

✓

```
30
```

✓

```
true
```

✓

```
["a", "b", "c"]
```

✓

```
null
```

✓

Some **invalid** JSON files

```
{'name': 'John'}
```

✗

```
{
  "name": "John"   // a person
}
```

✗

```
{"name": John}
```

✗

```
{
  "name": "John",
  "age": 30,
}
```

✗

```
{
  "name": "John"
  "age": 30
}
```

✗

```
{"ok": True}
```

✗

some errors to avoid:

✗

```
userObject: {
  name: "John"
}
```

`userObject` must be in quotes

✗

```
{ "name": "John" }: "value"
```

`objects` cannot be keys

✗

```
{ "name" }
```

inside an object, each entry must be a pair

```
{
  "...": {
    "...": "...",
    "...": {
      "...": "...",
      "...": {
        "...": "..."
      }
    }
  }
}
```

```
{
  "...": {
    "...": "...",
    "...": {
      "...": [
        {
          "...": "..."
        }
      ]
    }
  }
}
```

recursive, so structure can become complicated ...

```
{
  "...": {
    "...": "...",
    "...": 0,
    "...": true,
    "...": null,
    "...": {
      "...": "..."
    },
    "...": [
      "...",
      "..."
    ],
    "...": [
      {
        "...": "..."
      },
      {
        "...": "..."
      }
    ]
  }
}
```

start of VeloAccidents json file


```
{
  "type": "object",
  "required": [],
  "properties": {
    "Num_Acc": {
      "type": "string",
      "description": "Numero d'identification de l'accident",
      "x-capabilities": {
        "insensitive": false,
        "textAgg": false,
        "text": false,
        "textStandard": false,
        "values": false
      },
      "title": "Identifiant accident ",
      "x-group": "Identifiant de l'accident ",
      "x-transform": {
        "expr": "",
        "examples": [
          ""
        ],
        "type": "string"
      }
    },
    "date": {
      "type": "string",
      "description": "Date de l'accident",
      "x-capabilities": {
        "textStandard": false
      },
      "title": "Date de l'accident ",
      "x-group": "Date de l'accident ",
      "format": "date",
      "x-refersTo": "http://schema.org/Date",
      "x-concept": {
        "id": "date",
        "title": "Date d'evenement",
        "primary": true
      },
      "x-cardinality": 6840
    },
```


new idea

JSON in PostgreSQL

When to use JSON columns?

- nested or optional data whose shape varies
- data that arrives and leaves as JSON (APIs)
- you accept no data checking inside the JSON

When not?

- data you filter, join, or aggregate heavily
- data that needs foreign keys or constraints

```
CREATE TABLE users (
    name TEXT,
    profile JSONB
);

INSERT INTO users (name, profile) VALUES
('firstname', '{"name": "John", "age": 30, "address": {"city": "NYC", "zip": "10001"}}'),
('secondname', '{"name": "Alice", "age": 25, "address": {"city": "Boston", "zip": "02108"}}');
```

users

name	*profile*
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}

`json`: exact text copy, no indexing.

`jsonb`: binary, supports indexing.

Almost always use jsonb!

JSONB columns are **not validated**:

`"age": "banana"` stores fine!

The database cannot enforce types, required keys, or foreign keys inside the JSON.

users

name	*profile*
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}

```
SELECT profile->'name'
FROM users;
```

?column?
"John"
"Alice"

```
SELECT profile->'age'
FROM users;
```

?column?
30
25

```
SELECT profile->'address'
FROM users;
```

?column?
{"zip": "10001", "city": "NYC"}
{"zip": "02108", "city": "Boston"}

```
SELECT profile->'zip'
FROM users;
```

?column?

note: the returned column has JSONB type

users

name	*profile*
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}

```
SELECT profile->'zip'
FROM users;
```

?column?

```
SELECT profile->'address'->'zip'
FROM users;
```

?column?
"10001"
"02108"

-> : returns a JSONB type

users

name	*profile*
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}

```
SELECT profile->>'name'
FROM users;
```

?column?
John
Alice

```
SELECT profile->>'address'
FROM users;
```

?column?
{"zip": "10001", "city": "NYC"}
{"zip": "02108", "city": "Boston"}

->> : returns a TEXT type

users

name	*profile*
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}

->> : returns a TEXT type

```
SELECT profile->>'age' * 2.0
FROM users;
```

```
myuniversitydb=# SELECT profile->>'age' * 2.0
                 FROM users;
ERROR:  invalid input syntax for type numeric: "age"
LINE 1: SELECT profile->>'age' * 2.0
                         ^
```

```
SELECT (profile->>'age')::NUMERIC * 2.0
FROM users;
```

?column?
60.0
50.0

->> gives TEXT: cannot do arithmetic before casting to NUMERIC

users

name	*profile*
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}

```
SELECT pg_typeof(profile->'name'),
       pg_typeof(profile->>'name'),
       pg_typeof(profile->'address'),
       pg_typeof(profile->>'address'),
       pg_typeof(ARRAY[1,2,3])
FROM users
LIMIT 1;
```



pg_typeof	*pg_typeof*	*pg_typeof*	*pg_typeof*	*pg_typeof*
jsonb	text	jsonb	text	integer[]

```
SELECT *
FROM users
WHERE profile->'name' = 'John';
```



```
myuniversitydb=# SELECT *
myuniversitydb-# FROM users
myuniversitydb-# WHERE profile->'name' = 'John';
ERROR:  invalid input syntax for type json
LINE 3: WHERE profile->'name' = 'John';
                              ^
DETAIL:  Token "John" is invalid.
CONTEXT:  JSON data, line 1: John
```

```
SELECT *
FROM users
WHERE profile->>'name' = 'John';
```



name	*profile*
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}

JSON objects can be different across rows

```
INSERT INTO users (name, profile)
VALUES ('thirdname', '{"hobbies":["sports", "games", "reading"]}');
```

users

name	*profile*
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}
thirdname	{"hobbies": ["sports", "games", "reading"]}

```
SELECT profile->'hobbies'
FROM users;
```



?column?
null
null
["sports", "games", "reading"]

users

name	*profile*
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}
thirdname	{"hobbies": ["sports", "games", "reading"]}

Accessing array elements in JSON attributes

```
SELECT profile->'hobbies'->0, profile->'hobbies'->2
FROM users;
```



?column?	*?column?*
"sports"	"reading"

SQL arrays start at index **1**, JSON arrays start at index **0**

```
SELECT profile->>'name' as name
FROM users
WHERE profile @> '{"address": {"city": "NYC"}}';
```

@>: contains

name
John

? : has key

```
SELECT *
FROM users
WHERE profile ? 'hobbies' ;
```

name	profile
thirdname	{"hobbies": ["sports", "games", "reading"]}

? : has all keys in array

```
SELECT *
FROM users
WHERE profile ?& array['name', 'age'] ;
```

name	profile
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}

?| : has some keys in array

```
SELECT *
FROM users
WHERE profile ?| array['hobbies','what'] ;
```

name	profile
thirdname	{"hobbies": ["sports", "games", "reading"]}

users

name	profile
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}

```
SELECT jsonb_pretty(profile)
FROM users;
```

output is a text, not a table

```
myuniversitycb # SELECT jsonb_pretty(profile) FROM users;
       jsonb_pretty
--------------------------
 {                       +
     "age": 30,          +
     "name": "John",     +
     "address": {        +
         "zip": "10001", +
         "city": "NYC"   +
     }                   +
 }
 {                       +
     "age": 25,          +
     "name": "Alice",    +
     "address": {        +
         "zip": "02108", +
         "city": "Boston"+
     }                   +
 }
(2 rows)
```

users

name	*profile*
firstname	{"age": 30, "name": "John", "address": {"zip": "10001", "city": "NYC"}}
secondname	{"age": 25, "name": "Alice", "address": {"zip": "02108", "city": "Boston"}}
thirdname	{"hobbies": ["sports", "games", "reading"]}

```
SELECT jsonb_array_elements(profile->'hobbies')
FROM users;
```



jsonb_array_elements
"sports"
"games"
"reading"

teacher

id	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000.00
83821	Brandt	Comp. Sci.	92000.00
12121	Wu	Finance	90000.00
33456	Gold	Physics	87000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00
45565	Katz	Comp. Sci.	75000.00
76766	Crick	Biology	72000.00
10101	Srinivasan	Comp. Sci.	65000.00
58583	Califieri	History	62000.00
32343	El Said	History	60000.00
15151	Mozart	Music	40000.00

creating an array of JSON objects

```
SELECT dept_name,
       jsonb_agg( jsonb_build_object('name', name, 'salary', salary) )
FROM teacher
GROUP BY dept_name;
```


dept_name	*jsonb_agg*
Finance	[{"name": "Singh", "salary": 80000.00}, {"name": "Wu", "salary": 90000.00}]
History	[{"name": "Califieri", "salary": 62000.00}, {"name": "El Said", "salary": 60000.00}]
Physics	[{"name": "Einstein", "salary": 95000.00}, {"name": "Gold", "salary": 87000.00}]
Music	[{"name": "Mozart", "salary": 40000.00}]
Comp. Sci.	[{"name": "Srinivasan", "salary": 65000.00}, {"name": "Katz", "salary": 75000.00}, {"name": "Brandt", "salary": 92000.00}]
Biology	[{"name": "Crick", "salary": 72000.00}]
Elec. Eng.	[{"name": "Kim", "salary": 80000.00}]

update/modify `JSON` attribute value with jsonb_set and jsonb_insert

```
UPDATE users
SET profile = jsonb_set(profile, '{address, city}', '"Paris"')
WHERE name = 'firstname';
```

`->`	value as JSONB	`->>`	value as TEXT
`#>`	deep as JSONB	`#>>`	deep as TEXT
`@>`	contains	`<@`	contained
`?`	has key	`?&`	has all keys
`?|`	has any key	`||`	merge objects

JSON gives flexibility, but you lose:

—type checking, constraints and foreign keys *inside* the JSON
—simple `WHERE` optimisation.

Rule of thumb: real columns for data you query or validate;

JSON for **flexible, nested, semi-structured** data that arrives as JSON.

JSON is the common way to store lists

Apps speak JSON end-to-end: store and return API data without converting